

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



OPERATING SYSTEM

PROJECT 2 – System Call

Giảng viên: Vũ Thị Mỹ Hằng
Lê Quốc Hòa

Nhóm sinh viên thực hiện: **Nhóm 8**

22120008	Phạm Thiên An
22120071	Phan Bá Đức
22120215	Nguyễn Thị Mỹ

Thành phố Hồ Chí Minh, ngày 24 tháng 11 năm 2024

MỤC LỤC

MỤC LỤC	2
I. THÔNG TIN NHÓM.....	3
1. Thông tin thành viên:	3
2. Phân công:	3
II. NỘI DUNG.....	4
1. Tracing:.....	4
2. Sysinfo:.....	9
TÀI LIỆU THAM KHẢO.....	12

I. THÔNG TIN NHÓM

1. Thông tin thành viên:

STT	Họ và tên	MSSV
1	Phạm Thiên An	22120008
2	Phạm Bá Đức	22120071
3	Nguyễn Thị Mỹ	22120215

2. Phân công:

STT	Họ và tên	MSSV	Phân công	Mức độ hoàn thành
1	Phạm Thiên An	22120008	Tracing	100%
			Nộp bài	100%
2	Phạm Bá Đức	22120071	Sysinfo	100%
			Tổng hợp code	100%
3	Nguyễn Thị Mỹ	22120215	Tracing	100%
			Tổng hợp báo cáo	100%

II. NỘI DUNG

System call là một cửa ngõ vào kernel, cho phép tiến trình trên tầng user yêu cầu kernel thực thi một vài tác vụ cho mình. Những dịch vụ này có thể là tạo một tiến trình mới (fork), thực thi I/O (read, write), hoặc tạo ra một pipe cho giao tiếp liên tiến trình (IPC).

1. Tracing:

1.1. Mô tả yêu cầu:

Hãy thêm 1 system call tên là **trace** để điều khiển theo dõi. Khi gọi system call này nó sẽ in ra mỗi dòng thông tin khi có system call bị theo dõi được gọi gồm có: id của tiến trình gọi system call này, tên system call được gọi và giá trị trả về của nó. System call **trace** sẽ bật theo dõi cho tiến trình gọi nó và bất kỳ tiến trình con nào mà nó phân nhánh sau đó, nhưng không được ảnh hưởng đến các quy trình khác.

1.2. Phương pháp thực hiện:

1.2.1. Ý tưởng bài toán:

- System call trace sẽ nhận 1 đối số **trace_mask** từ người dùng. Những số bit của nó sẽ xác định xem system call nào sẽ được theo dõi.

1.2.2. Các bước thực hiện:

a. Thêm system call trace:

- Thêm **prototype** cho system call vào file user.h: Điều này giúp trình biên dịch có thể nhận diện lời gọi hàm từ chương trình người dùng, tạo một liên kết rõ ràng giữa không gian người dùng và nhân hệ điều hành.
- Thêm **stub** vào user/usys.pl: Để hệ thống biết cách chuyển đổi từ lời gọi hàm trong không gian người dùng sang lời gọi system call trong nhân hệ điều hành.
- Thêm **syscall number** vào file kernel/syscall.h: Giúp hệ thống nhận diện được system call đó. Mỗi system call sẽ có 1 syscall number duy nhất đại diện cho nó.
- Thêm **system call** mới vào kernel/syscall.c: Nhằm liên kết syscall number với hàm xử lý tương ứng trong kernel.
- Thêm hàm **sys_trace** vào kernel/sysproc.c: Để thực hiện lệnh gọi hệ thống mới bằng cách ghi nhớ đối số của nó trong một biến mới trong cấu trúc proc.

*b. Cập nhật cấu trúc **proc**:*

- Thêm biến mới **traced** để làm dấu các system call được theo dõi trong mỗi tiến trình.

*c. Điều chỉnh hàm **fork** trong **kernel/proc.c**:*

- Nhằm đảm bảo **traced** cũng được kế thừa bởi tiến trình con, ta sao chép giá trị **traced** của tiến trình cha sang tiến trình con trong quá trình tạo ra nó.

*d. Điều chỉnh hàm **syscall** trong **kernel/syscall.c**:*

- Kiểm tra xem system call có đang được theo dõi không dựa trên **traced** của tiến trình hiện tại.
- Nếu có, ta sẽ in ra các thông tin chi tiết của system call đó theo yêu cầu đề bài.

e. Thực hiện Challenge của đề bài:

- Bổ sung các hàm để trả về tên và số lượng tham số của các system call.
- Điều chỉnh hàm **syscall** trong **kernel/syscall.c**:
 - + Thêm việc lấy tham số của các system call.
 - + Bổ sung các tham số của system call vào kết quả được in ra.
- Kết quả: In ra các tham số của các system call được theo dõi.

1.3. Kết quả thực hiện:

- Thực hiện các test case của đề bài:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read(3, 4112, 1023) -> 1023
3: syscall read(3, 4164, 971) -> 971
3: syscall read(3, 4174, 961) -> 298
3: syscall read(3, 4112, 1023) -> 0
$
$
$
$
$ trace 2147483647 grep hello README
8: syscall trace(2147483647) -> 0
8: syscall exec(16336, 15952) -> 3
8: syscall open(16336, 0) -> 3
8: syscall read(3, 4112, 1023) -> 1023
8: syscall read(3, 4164, 971) -> 971
8: syscall read(3, 4174, 961) -> 298
8: syscall read(3, 4112, 1023) -> 0
8: syscall close(3) -> 0
$
$ grep hello README
$
```

```
$ trace 2 usertests forkforkfork
usertests starting
12: syscall fork() -> 13
test forkforkfork: 12: syscall fork() -> 14
14: syscall fork() -> 15
15: syscall fork() -> 16
15: syscall fork() -> 17
16: syscall fork() -> 18
16: syscall fork() -> 19
16: syscall fork() -> 20
15: syscall fork() -> 21
16: syscall fork() -> 22
16: syscall fork() -> 23
16: syscall fork() -> 24
16: syscall fork() -> 25
17: syscall fork() -> 26
15: syscall fork() -> 27
15: syscall fork() -> 28
15: syscall fork() -> 29
15: syscall fork() -> 30
15: syscall fork() -> 31
15: syscall fork() -> 32
15: syscall fork() -> 33
15: syscall fork() -> 34
15: syscall fork() -> 35
15: syscall fork() -> 36
15: syscall fork() -> 37
15: syscall fork() -> 38
15: syscall fork() -> 39
15: syscall fork() -> 40
15: syscall fork() -> 41
15: syscall fork() -> 42
16: syscall fork() -> 43
16: syscall fork() -> 44
```

```
15: syscall fork() -> 42
16: syscall fork() -> 43
16: syscall fork() -> 44
17: syscall fork() -> 45
15: syscall fork() -> 46
15: syscall fork() -> 47
16: syscall fork() -> 48
16: syscall fork() -> 49
16: syscall fork() -> 50
15: syscall fork() -> 51
15: syscall fork() -> 52
15: syscall fork() -> 53
15: syscall fork() -> 54
15: syscall fork() -> 55
15: syscall fork() -> 56
15: syscall fork() -> 57
15: syscall fork() -> 58
15: syscall fork() -> 59
15: syscall fork() -> 60
17: syscall fork() -> 61
17: syscall fork() -> 62
17: syscall fork() -> 63
15: syscall fork() -> 64
16: syscall fork() -> 65
16: syscall fork() -> 66
16: syscall fork() -> 67
16: syscall fork() -> 68
16: syscall fork() -> 69
16: syscall fork() -> 70
16: syscall fork() -> 71
16: syscall fork() -> 72
15: syscall fork() -> 73
15: syscall fork() -> 74
15: syscall fork() -> -1
17: syscall fork() -> -1
```

```
15: syscall fork() -> 73
15: syscall fork() -> 74
15: syscall fork() -> -1
17: syscall fork() -> -1
OK
12: syscall fork() -> 75
ALL TESTS PASSED
$
```


2. Sysinfo:

2.1. Mô tả yêu cầu:

- **Mục tiêu:** Cập nhật hệ thống bằng cách thêm một hệ thống gọi mới, sysinfo, nhằm thu thập các thông tin về hệ thống đang chạy, bao gồm:
 - + Số lượng bộ nhớ trống (freemem).
 - + Số lượng tiến trình đang hoạt động (n proc).
 - + Tính toán và xuất ra giá trị Load Average.
- **Yêu cầu:** Hệ thống gọi **sysinfo** cần nhận một con trỏ đến cấu trúc sysinfo, sau đó điền thông tin về bộ nhớ trống, số tiến trình đang chạy và Load Average vào các trường tương ứng trong cấu trúc đó

2.2. Phương pháp thực hiện:

2.2.1. Hàm free_memory

- **Mục tiêu:** Đếm tổng số bộ nhớ trống hiện có trong hệ thống.
- **Cách thực hiện:**
 - + Duyệt qua danh sách các trang bộ nhớ (freelist) để tính tổng bộ nhớ còn lại.
 - + Mỗi trang bộ nhớ có kích thước PGSIZE, vì vậy mỗi phần tử trong freelist đại diện cho một lượng bộ nhớ tương ứng.
 - + Sử dụng cơ chế đồng bộ hóa với khóa (lock) để tránh tranh chấp khi truy cập freelist.

2.2.2. Hàm getnproc

- **Mục tiêu:** Đếm số lượng tiến trình không có trạng thái UNUSED.
- **Cách thực hiện:**
 - + Duyệt qua tất cả các tiến trình trong hệ thống và kiểm tra trạng thái của chúng.
 - + Nếu trạng thái không phải UNUSED, tăng số đếm.

2.2.3. Hàm get_loadavg

- **Mục tiêu:** Tính toán Load Average.
- **Cách thực hiện:**

- + Load Average được tính bằng tỷ lệ giữa số lượng tiến trình RUNNABLE và số lần cập nhật (ticks) trong một khoảng thời gian.
- + Để tránh chia cho 0, chỉ thực hiện tính toán khi runnable_ticks > 0.
- + Load Average được nhân với 100 để đưa ra tỷ lệ phần trăm.

2.2.4. Hàm sys_sysinfo

- **Mục tiêu:** Hệ thống gọi này nhận địa chỉ của cấu trúc sysinfo, điền thông tin vào cấu trúc đó và sao chép lại thông tin cho người dùng.
- **Cách thực hiện:**
 - + Nhận địa chỉ của cấu trúc sysinfo từ người dùng qua argaddr.
 - + Điền các giá trị vào trường của cấu trúc (freemem, nproc, loadavg).
 - + Sao chép cấu trúc này trở lại không gian người dùng bằng copyout.

2.3. Kết quả thực hiện:

- Kết quả sau khi thực hiện lệnh **sysinfotest**:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sysinfotest
sysinfotest: start
sysinfotest: OK
Free memory: 132988928 bytes
Number of processes: 3
Load average (x100): 9
$
```

2.4. Khó khăn:

- Khó khăn trong việc xác định các sử dụng của các hàm dùng trong bài như: **copyout**, **argaddr**, **acquire**,... và các cấu trúc khác
- Khó khăn trong việc tìm công thức **tính Load Average** trên internet. Nên em dùng chatGPT thì tìm được hai công thức:

○ Công thức EWMA (exponentially weighted moving average)

$$LA_t = LA_{t-1} \cdot e^{\frac{-1}{T}} + n \cdot (1 - e^{\frac{-1}{T}})$$

Trong đó:

- LA_t : Load Average tại thời điểm t .
- LA_{t-1} : Load Average tại thời điểm trước đó.
- T : Khoảng thời gian (ví dụ: 1 phút, 5 phút, hoặc 15 phút).
- n : Số tiến trình **RUNNABLE** tại thời điểm t .

○ Công thức đơn giản hóa

$$LA = \frac{\text{Tổng số tiến trình RUNNABLE trong các lần đo}}{\text{Số lần đo}}$$

Ban đầu em định tính theo công thức **EWMA** nhưng lại gặp khó khăn trong việc đồng bộ hóa các biến dùng chung giữa các tiến trình. Mặc dù đã cố sử dụng **acquire** và **release** để khóa khi cập nhật nhưng vẫn bị lỗi dù đã thử nhiều cách. Và điều thứ hai là dù thử nhiều cách nhưng có vẻ xv6 không có kiểu dữ liệu double vì em đã chạy với kiểu dữ liệu là double thì bị lỗi, thay kiểu double bằng uint64 thì chạy được. Vì vậy em quyết định dùng công thức thứ hai đơn giản hơn.

TÀI LIỆU THAM KHẢO

- [1] (n.d.). Retrieved from https://courses.fit.hcmus.edu.vn/pluginfile.php/214961/-mod_resource/content/1/Lab02%20System%20Call%20En.pdf
- [2] Nguyễn, T. (n.d.). *System Call*. Retrieved from Studocu: <https://www.studocu.com/vn/document/dai-hoc-ton-duc-thang/he-dieu-hanh/system-call/61441061>
- [3] YSY, S. (n.d.). *MIT 6.1810: Lab 2 - System Calls Documentation*. Retrieved from Github: https://github.com/sakura-ysy/MIT6.1810/blob/main/doc/Lab2-Syste-m_Calls/lab2.md